

Python for Data Science: A Beginner's Guide

Programming thinking & Python basics · Python 3 · VS Code

This guide takes you from your very first line of Python to a small working program. Each chapter introduces one idea, shows it in a short piece of code, and explains what happened. The examples use the kind of values data scientists handle every day: sales figures, prices, counts, names and measurements.

Type the examples yourself rather than copying them. Every example was run on Python 3 and the output shown is the real output. Error wording can differ slightly between Python versions; the error type stays the same.

Chapter	Topic	You will learn to
1	Thinking like a programmer	Break a problem into inputs, steps and outputs
2	Running Python	Run code in VS Code, the terminal and the REPL
3	Variables and statements	Store values under names and update them
4	Data types	Tell int, float, str and bool apart
5	Operators	Calculate, compare and combine True/False values
6	Input and type conversion	Read from the keyboard and turn text into numbers
7	Working with strings	Clean text, pick characters and format output
8	Errors and debugging	Read an error message and fix the cause
9	Putting it together	Write a complete small program

Chapter 1 Thinking like a programmer

A computer does exactly what it is told, in the order it is told, and nothing more. It cannot guess a missing step the way a colleague can. So before writing any code, a programmer describes the problem precisely.

Take a simple request from a store manager: *"Tell me today's revenue and whether we hit our target."* To a person that is clear enough. To a computer it raises questions. Revenue from what? What is the target? What should the answer look like?

A model for every problem



Stage	Question to ask	For the store manager
Problem	What needs to be solved?	Report today's revenue against target
Inputs	What facts must be supplied?	units sold, unit price, daily target
Steps / Rules	What happens to them, in what order?	revenue = units × price; compare revenue with target
Output	What should the user see?	the revenue, and True/False for target met
Test / Correct	How do we know it is right?	work out one case by hand and compare

Written as plain steps, this is an **algorithm**: a precise sequence of instructions. Python is simply a language for writing algorithms in a form a computer can run.

Where Python fits

Tool	How it expresses logic
Spreadsheet	A formula inside a cell, e.g. <code>=B2*C2</code>
SQL	A query that describes which data to retrieve or summarise
Python	An ordered list of instructions: create values, transform them, compare them, show results

In data science: Almost every analysis follows this model. Data comes in (inputs), it is cleaned and transformed (steps), results are reported (output), and the numbers are checked before anyone acts on them (test).

Key points

- Decide inputs, steps and outputs before writing code.
- An algorithm is a precise, ordered set of steps.
- Always test with a case you can check by hand.

TRY IT YOURSELF

A sensor records a temperature of 25 degrees Celsius. You want it in Fahrenheit using $^{\circ}\text{F} = ^{\circ}\text{C} \times 9 / 5 + 32$. Write down the input, the steps and the output, then work out the answer by hand.

Chapter 2 Running Python

Three words are often confused. Keeping them apart will save you a lot of confusion.

Language

Python: the rules for writing instructions, and the interpreter that carries them out.

Runtime

A running Python process plus its installed packages. VS Code calls it the *interpreter* and shows it in the bottom-right corner.

Interface

Where you write and run code: a `.py` file, the terminal, or a notebook (`.ipynb`).

You will hear other names too. **Anaconda** is a popular way to install Python together with data-science tools. **Jupyter** and **Google Colab** are notebook interfaces, where code lives in cells. None of them is a different language: the same Python code gives the same result everywhere.

Your first program

In VS Code, open a folder (**File > Open Folder**), create a file called `hello.py`, type this line, save, and click the **Run Python File** button:

`hello.py`

```
print("Hello, data science!")
```

OUTPUT

```
Hello, data science!
```

`print()` displays whatever you put between its brackets. A file like this is called a **script**. When you run it, Python starts at line 1 and works down to the last line.

You can also run it from the VS Code terminal (**Terminal > New Terminal**):

TERMINAL

```
> python hello.py
Hello, data science!
```

Watch out: On Windows the command may be `py` instead of `python`; on macOS it may be `python3`. Use whichever works on your machine.

A script only shows what you print

`ch2_print.py`

```
10 + 5
print(10 + 5)
```

OUTPUT

```
15
```

Both lines calculate 15, but only one number appears. In a script, a result is thrown away unless you `print()` it. Notebooks show the value of the last line automatically, which is why code copied from a notebook into a script can seem to do nothing.

The REPL: one line at a time

Type `python` in the terminal. The `>>>` prompt is the **REPL** (Read, Evaluate, Print, Loop). It runs each line as you type it and shows the result straight away, which is handy for quick experiments. Type `exit()` to leave.

TERMINAL (REPL)

```
>>> 10 + 5
15
>>> units = 120
>>> units * 2
240
>>> exit()
```

Notice that the REPL **remembered** `units` from one line to the next. A notebook behaves the same way: it remembers everything you have run, in the order you ran it, not the order on the page. A script forgets everything when it ends and starts fresh each time.

In data science: Notebooks are great for exploring data step by step; scripts are how finished analysis runs reliably, for example on a schedule. Before sharing a notebook, restart it and run every cell from the top, to prove it does not depend on something you ran earlier.

Key points

- Python is the language; the runtime runs it; the interface is where you type it.
- A script runs top to bottom, starting fresh every time.
- In a script, use `print()` to see a result.
- The REPL and notebooks remember values in the order you ran them.

Chapter 3 Variables and statements

A **variable** is a name that refers to a value, so you can use the value later without retyping it.

`ch3_var.py`

```
units_sold = 120
print(units_sold)
```

OUTPUT

```
120
```

Piece	Called	Meaning
<code>units_sold</code>	variable name	the label you chose
<code>=</code>	assignment operator	store the value on the right under the name on the left
<code>120</code>	literal	a value written directly in the code
the whole line	statement	one instruction for Python to carry out

Read the line as "**assign 120 to units_sold**", not "units_sold equals 120". Python works out the right-hand side first and then attaches the result to the name.

Expressions

An **expression** is any piece of code that produces a value. Here, `units_sold * unit_price` is an expression; its result is stored in `revenue`.

`ch3_expr.py`

```
units_sold = 120
unit_price = 45.5
revenue = units_sold * unit_price
print(revenue)
```

OUTPUT

```
5460.0
```

Changing a variable

Assigning to a name again replaces its value. Because the right side is worked out first, a variable can even be updated using its own value:

ch3_update.py

```
units_sold = 120
print(units_sold)

units_sold = units_sold + 30
print(units_sold)
```

OUTPUT

```
120
150
```

Python reads the current value (120), adds 30, and stores 150 back under the same name. Order matters: a line only sees the values that exist when it runs.

Names and text are different things

ch3_text.py

```
city = "Johannesburg"
print(city)
print("city")
```

OUTPUT

```
Johannesburg
city
```

Without quotes, `city` is a name and Python looks up its value. With quotes, `"city"` is just the text c-i-t-y.

Naming rules

Rule	Good	Not allowed
letters, digits, underscores only	<code>unit_price</code>	<code>unit price</code> , <code>unit-price</code>
cannot start with a digit	<code>q1_sales</code>	<code>1q_sales</code>
case-sensitive	<code>sales</code> and <code>Sales</code> are different names	
not a Python keyword	<code>class_label</code>	<code>class</code> , <code>True</code> , <code>if</code>

Python programmers use **snake_case**: lowercase words joined by underscores. Choose names that say what the value means. `revenue = units_sold * unit_price` explains itself; `r = a * b` does not.

Comments and indentation

ch3_comments.py

```
# Revenue before any discount
revenue = 120 * 45.5
print(revenue) # comments can also follow code
```

OUTPUT

```
5460.0
```

Python ignores everything after `#`. Use comments to explain *why* you did something. Also notice that every line starts at the left margin. In Python, spaces at the start of a line have meaning, so an accidental indent causes an error (Chapter 8).

Key points

- `=` assigns: right side first, then store it under the name.
- Assigning again replaces the old value.
- Use clear snake_case names; Python is case-sensitive.
- `#` starts a comment; keep statements at the left margin.

TRY IT YOURSELF

Predict the output, then run it: `visits = 50 / visits = visits * 2 / print(visits) / print("visits")`

Chapter 4 Data types

Every value in Python has a **type**. The type decides what you can do with the value. Four types cover everything in this guide:

Type	Holds	Data science example
<code>int</code>	whole numbers	number of customers, units sold
<code>float</code>	decimal numbers	price, temperature, conversion rate
<code>str</code>	text, written in quotes	customer name, region, product code
<code>bool</code>	<code>True</code> or <code>False</code>	did we hit target? is the reading valid?

`type()` tells you the type of any value:

`ch4_types.py`

```
customers = 340
avg_spend = 212.75
region = "Gauteng"
target_met = True

print(type(customers))
print(type(avg_spend))
print(type(region))
print(type(target_met))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Read `<class 'int'>` as "this value is an int". You do not need to know what *class* means yet.

Why types matter

ch4_why.py

```
print(4 * 10)
print("4" * 10)
```

OUTPUT

```
40
4444444444
```

4 is a number; "4" is text that happens to contain the character 4. Multiplying a number does maths. Multiplying text **repeats** it. Python gave no error, just a very different answer. With a decimal it does refuse:

ch4_err.py

```
print("4" * 10.0)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\ch4_err.py", line 1, in <module>
    print("4" * 10.0)
    ~~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'
```

In data science: Data often arrives as text: from a CSV file, a web form or a user. A column of numbers stored as text is one of the most common causes of wrong results, so checking types is a habit worth building now.

True and False

True and False are written with a capital letter and no quotes. "False" in quotes is just a five-letter word, not a boolean.

ch4_bool.py

```
print(type(False))
print(type("False"))
```

OUTPUT

```
<class 'bool'>
<class 'str'>
```

A name can change type

ch4_dynamic.py

```
reading = 25
print(type(reading))
reading = "twenty-five"
print(type(reading))
```

OUTPUT

```
<class 'int'>
<class 'str'>
```

Python does not lock a name to one type. This is flexible, but reusing a name for a different kind of value makes code confusing. Keep each name for one kind of value.

Key points

- Four core types: `int`, `float`, `str`, `bool`.
- `type()` shows a value's type.
- 4 and "4" look alike but behave differently.
- `True/False` have capitals and no quotes.

TRY IT YOURSELF

What is the type of each value? `3`, `3.0`, `"3"`, `3 > 2`. Check your answers with `type()`.

Chapter 5 Operators

Operators are the symbols that calculate and compare values.

Arithmetic

Operator	Meaning	Example	Result
<code>+</code>	add	<code>120 + 30</code>	<code>150</code>
<code>-</code>	subtract	<code>5000 - 4200</code>	<code>800</code>
<code>*</code>	multiply	<code>120 * 45.5</code>	<code>5460.0</code>
<code>/</code>	divide	<code>80 / 100</code>	<code>0.8</code>

`ch5_arith.py`

```
print(120 + 30)
print(120 * 45.5)
print(10 / 2)
```

OUTPUT

```
150
5460.0
5.0
```

Two rules to notice. An `int` combined with a `float` gives a `float`. Division with `/` always gives a `float`, even when the answer is whole (`10 / 2` is `5.0`).

Order of operations and brackets

Python follows the usual maths order: `*` and `/` before `+` and `-`. Brackets make your intention clear:

`ch5_brackets.py`

```
yesterday = 4000.0
today = 5460.0
growth_pct = (today - yesterday) / yesterday * 100
print(growth_pct)
```

OUTPUT

```
36.5
```

Without the brackets, Python would divide `yesterday` by itself first and give a meaningless answer. When an expression gets long, break it into named steps instead.

`ch5_steps.py`

```
change = today - yesterday
growth_pct = change / yesterday * 100
print(change)
print(growth_pct)
```

Comparisons

A comparison asks a question and the answer is a `bool`.

Operator	Asks
<code>==</code>	equal?
<code>!=</code>	not equal?
<code>></code> <code><</code>	greater? less?
<code>>=</code> <code><=</code>	greater or equal? less or equal?

`ch5_compare.py`

```
revenue = 5460.0
target = 5000.0

print(revenue >= target)
print(revenue == target)
print(revenue != target)
```

OUTPUT

```
True
False
True
```

= and == are not the same

Code	Meaning
<code>target = 5000.0</code>	Assignment. An instruction: store 5000.0 under <code>target</code> .
<code>target == 5000.0</code>	Comparison. A question: is <code>target</code> equal to 5000.0? Gives <code>True</code> or <code>False</code> .

and, or, not

These combine `True/False` values.

ch5_logic.py

```

revenue = 5460.0
returns = 3

target_met = revenue >= 5000
few_returns = returns < 5

print(target_met and few_returns)
print(target_met or few_returns)
print(not target_met)

```

OUTPUT

```

True
True
False

```

Operator	Result is True when
and	both sides are True
or	at least one side is True
not	flips True to False and False to True

In data science: Comparisons and `and`/`or`/`not` are how you will later filter data, for example keeping only the rows where sales are above target *and* the region is Gauteng.

Key points

- `+` `-` `*` `/` do arithmetic; `/` always gives a float.
- Use brackets to show the order you mean.
- Comparisons return `True` or `False`.
- `=` assigns; `==` compares.

TRY IT YOURSELF

Using the formula from Chapter 1, write a script that stores `celsius = 25`, calculates `fahrenheit`, prints it, and prints whether `fahrenheit` is greater than 75.

Chapter 6 Input and type conversion

Programs often need values from outside the code. The simplest source is the keyboard, using `input()`.

ch6_input.py

```
units_text = input("Units sold: ")
print(units_text)
print(type(units_text))
```

TERMINAL

```
> python ch6_input.py
Units sold: 120
120
<class 'str'>
```

The program shows the prompt and waits. The user typed `120`, but the program received the **text** `"120"`. `input()` always returns a `str`.

Converting between types

Function	Converts to	Example
<code>int()</code>	a whole number	<code>int("120") → 120</code>
<code>float()</code>	a decimal number	<code>float("45.5") → 45.5</code>
<code>str()</code>	text	<code>str(120) → "120"</code>

ch6_convert.py

```
units = int(input("Units sold: "))
price = float(input("Unit price: "))
revenue = units * price
print("Revenue:", revenue)
```

TERMINAL

```
> python ch6_convert.py
Units sold: 120
Unit price: 45.5
Revenue: 5460.0
```

Read a line like `units = int(input("Units sold: "))` **from the inside out**: first `input()` collects the text, then `int()` converts it, then `=` stores the result.

Turning numbers into text

`+` joins text to text. To join a number to text, convert it first:

ch6_str.py

```
units = 120
print("Units: " + str(units))
print("Units: " + units)
```

OUTPUT

```
Units: 120
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\ch6_str.py", line 3, in <module>
    print("Units: " + units)
                    ~~~~~^~~~~~
TypeError: can only concatenate str (not "int") to str
```

When conversion fails

If the text cannot be read as a number, Python raises a **ValueError**:

ch6_bad.py

```
units = int(input("Units sold: "))
print(units)
```

TERMINAL

```
> python ch6_bad.py
Units sold: twelve
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\ch6_bad.py", line 1, in <module>
    units = int(input("Units sold: "))
ValueError: invalid literal for int() with base 10: 'twelve'
```

`int()` also refuses decimals: `int("12.5")` is a `ValueError` too. Use `float()` for values that may have a decimal point.

In data science: Converting text to the right type is a daily task: reading a CSV, every value may arrive as text. Later in the course, Pandas will do much of this for you, but the idea is exactly the same.

Key points

- `input()` always returns text.
- Convert with `int()` or `float()` before doing maths.
- Use `str()` to join a number to text.
- Text that is not a number causes a `ValueError`.

TRY IT YOURSELF

Write a script that asks for a temperature in Celsius (it may have decimals), converts it, and prints the Fahrenheit value. Test with 25 and with 18.5.

Chapter 7 Working with strings

A **string** is text: any characters inside quotes. Real-world text is messy, so Python gives strings useful tools called **methods**, used with a dot after the value.

Cleaning text

ch7_clean.py

```
region = "  gauteng  "
print "[" + region + "]"
print "[" + region.strip() + "]"
print region.strip().title()
print region.strip().upper()
```

OUTPUT

```
[  gauteng  ]
[gauteng]
Gauteng
GAUTENG
```

Method	Does
<code>.strip()</code>	removes spaces at the start and end
<code>.title()</code>	capitalises the first letter of each word
<code>.upper()</code>	makes every letter a capital

The square brackets are only there to make the spaces visible. Methods can be **chained**; they run left to right, so `.strip().title()` strips first and then capitalises.

Watch out: A method gives back a new value; it does not change the original. To keep the cleaned version, assign it: `region = region.strip().title()`.

Length

ch7_len.py

```
region = "Gauteng"
print len(region)
print len("Cape Town")
```

OUTPUT

```
7
9
```

`len()` counts characters. Spaces count too, so "Cape Town" has 9.

Indexing: one character at a time

Each character has a position called an **index**. Python counts from 0, and negative indexes count back from the end.

G	a	u	t	e	n	g	character
0	1	2	3	4	5	6	from the start
-7	-6	-5	-4	-3	-2	-1	from the end

ch7_index.py

```
region = "Gauteng"
print(region[0])
print(region[1])
print(region[-1])
print(region[-2])
```

OUTPUT

```
G
a
g
n
```

Watch out: The first character is `[0]`, not `[1]`. Asking for a position beyond the end, such as `region[7]`, gives an `IndexError`.

Joining text

ch7_join.py

```
region = "Gauteng"
store_number = 7
store_code = region[0] + "-" + str(store_number)
print(store_code)
```

OUTPUT

```
G-7
```

`+` joins strings together. The number had to go through `str()` first.

f-strings: the readable way to format output

Put `f` before the opening quote and Python fills in anything between `{ }`:

ch7_fstring.py

```
region = "Gauteng"
revenue = 5460.0
growth_pct = 36.5

print(f"Region: {region}")
print(f"Revenue: {revenue:.2f}")
print(f"Growth: {growth_pct:.1f}%")
```

OUTPUT

```
Region: Gauteng
Revenue: 5460.00
Growth: 36.5%
```

`:.2f` shows two decimal places and `:.1f` shows one. This changes only how the number is **displayed**; the stored value is unchanged.

In data science: Text cleaning is a large part of real data work: names with stray spaces, inconsistent capitals and codes that must be split apart. `"gauteng"`, `"Gauteng "` and `"GAUTENG"` look the same to a person but are three different values to Python until you clean them.

Key points

- `.strip()`, `.title()`, `.upper()` clean text; assign the result to keep it.
- `len()` counts characters, including spaces.
- Indexing starts at 0; `[-1]` is the last character.
- f-strings with `:.2f` give readable output.

TRY IT YOURSELF

Start with `name = " siphon dlamini "`. Clean it to `Sipho Dlamini`, print its length, its first character and its last character.

Chapter 8 Errors and debugging

Everyone gets errors, every day. An error message is not a judgement; it is **evidence** that tells you what Python could not do and where. Learning to read it is one of the most valuable skills in programming.

Reading an error message

`ch8_trace.py`

```
revenue = 5460.0
print(revenu)
```

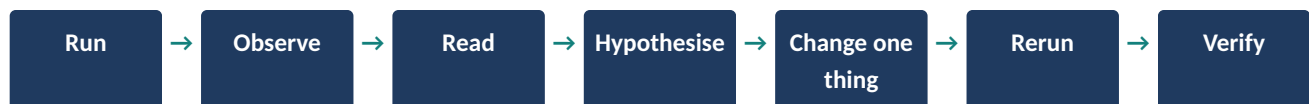
ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\ch8_trace.py", line 2, in <module>
    print(revenu)
          ^^^^^
NameError: name 'revenu' is not defined. Did you mean: 'revenue'?
```

This is a **traceback**. Read it from the **bottom up**:

1. **Last line:** the error type and a message: `NameError: name 'revenu' is not defined`.
2. **File line:** where it happened: line 2. In VS Code, click the path to jump there.
3. **Code line and ^ marks:** exactly which part Python objected to.

The debugging loop



Change **one** thing at a time. If you change several lines at once, you will not know which change fixed the problem.

Five errors you will meet first

err_syntax.py

```
print("Revenue report)
```

ERROR

```
File "C:\Users\student\s07_python_basics\err_syntax.py", line 1
  print("Revenue report)
      ^
SyntaxError: unterminated string literal (detected at line 1)
```

SyntaxError: Python cannot read the code. Here the closing quote is missing.

err_indent.py

```
units = 120
  price = 45.5
```

ERROR

```
File "C:\Users\student\s07_python_basics\err_indent.py", line 2
  price = 45.5
IndentationError: unexpected indent
```

IndentationError: unexpected spaces at the start of a line. Move it back to the left margin.

err_name.py

```
revenue = 5460.0
print(Revenue)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\err_name.py", line 2, in <module>
    print(Revenue)
          ^^^^^^^
NameError: name 'Revenue' is not defined. Did you mean: 'revenue'?
```

NameError: no variable with that name. Usually a typo (remember names are case-sensitive), or the line that creates it has not run yet.

err_type.py

```
units = "120"
revenue = units * 45.5
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\err_type.py", line 2, in <module>
    revenue = units * 45.5
              ~~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'
```

TypeError: the operation got the wrong kind of value, here text instead of a number. Convert with `int()` first.

err_value.py

```
price = float("forty")
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\err_value.py", line 1, in <module>
    price = float("forty")
ValueError: could not convert string to float: 'forty'
```

ValueError: the operation is right but this value cannot be used. `float()` cannot read the word forty.

Error	In plain words	Typical fix
<code>SyntaxError</code>	Python cannot read this line	check quotes and brackets
<code>IndentationError</code>	unexpected spaces at line start	move the line to the left margin
<code>NameError</code>	this name does not exist	fix the spelling, or run the line that creates it
<code>TypeError</code>	wrong kind of value	convert the type
<code>ValueError</code>	right kind, unusable value	fix the input value

Bugs without errors

The hardest bugs give no error at all: a `+` where you meant `-`, or forgotten brackets. The program runs and gives a wrong answer. The only defence is testing: compare results with a case you worked out by hand, and check intermediate values with `print()`.

In data science: An analysis that runs without errors is not necessarily correct. Checking totals and spot-checking values against the source, as you did in the SQL sessions, applies equally to Python.

Key points

- Read a traceback from the last line up.
- Name the error type before you change anything.
- Change one thing, then run again.
- No error does not mean correct: test with known answers.

TRY IT YOURSELF

Save the code below as `practice_debug.py`. Fix it one error at a time, running after each fix, and write down each error type.

practice_debug.py

```
units = "8"
price = 12.5
total = units * price
print(totl)
```

Chapter 9 Putting it together

Now combine everything into one program. A store manager types in today's figures and gets a short, clean report.

Plan first

Stage	For this program
Inputs	store name, region, units sold, unit price, yesterday's revenue; target fixed at 5000.0
Steps	clean the text; convert the numbers; revenue; growth %; target check; store code
Output	a formatted report
Test	Rosebank, Gauteng, 120 units at 45.50, yesterday 4000: revenue 5460.00, growth 36.5%, target met True, code G-R

sales_report.py

```

# Daily sales report

# Inputs (text from the keyboard)
store = input("Store name: ")
region = input("Region: ")
units = int(input("Units sold: "))
unit_price = float(input("Unit price: "))
yesterday = float(input("Yesterday's revenue: "))
target = 5000.0

# Clean the text
store = store.strip().title()
region = region.strip().title()

# Calculate
revenue = units * unit_price
growth_pct = (revenue - yesterday) / yesterday * 100
target_met = revenue >= target
store_code = region[0].upper() + "-" + store[0].upper()

# Report
print("DAILY SALES REPORT")
print(f"Store: {store} ({store_code})")
print(f"Region: {region}")
print(f"Revenue: {revenue:.2f}")
print(f"Growth vs yesterday: {growth_pct:.1f}%")
print(f"Target met: {target_met}")

```

TERMINAL

```

> python sales_report.py
Store name:  rosebank
Region: gauteng
Units sold: 120
Unit price: 45.50
Yesterday's revenue: 4000
DAILY SALES REPORT
Store: Rosebank (G-R)
Region: Gauteng
Revenue: 5460.00
Growth vs yesterday: 36.5%
Target met: True

```

Look at how the chapters appear in this one file:

Chapter	Where it shows up
1 Thinking	the plan above and the sections in the code
3 Variables	every <code>=</code> line
4 Types	<code>int</code> units, <code>float</code> prices, <code>str</code> names, <code>bool</code> target_met
5 Operators	revenue, growth % with brackets, <code>>=</code>
6 Input	<code>input()</code> wrapped in <code>int()</code> and <code>float()</code>
7 Strings	<code>.strip().title()</code> , <code>[0]</code> , <code>+</code> , f-strings
8 Errors	type <code>abc</code> for units and you will meet a <code>ValueError</code>

What comes next

This program runs the same steps every time, for one store. The next topics make it more powerful:

You might want to...	You will learn
print a different message when the target is missed	decisions with <code>if / elif / else</code>
repeat the report for many stores	loops
reuse the calculation	functions
handle bad input without crashing	<code>try / except</code>
keep many stores together	lists and dictionaries
analyse a whole table of sales	NumPy and Pandas

Answers to the exercises

Chapter 1

Input: 25 °C. **Steps:** multiply by 9, divide by 5, add 32. **Output:** the Fahrenheit value. **By hand:** $25 \times 9 = 225$; $225 / 5 = 45$; $45 + 32 = 77$ °F.

Chapter 3

a3.py

```
visits = 50
visits = visits * 2
print(visits)
print("visits")
```

OUTPUT

```
100
visits
```

Chapter 4

a4.py

```
print(type(3))
print(type(3.0))
print(type("3"))
print(type(3 > 2))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Chapter 5

a5.py

```
celsius = 25
fahrenheit = celsius * 9 / 5 + 32
print(fahrenheit)
print(fahrenheit > 75)
```

OUTPUT

```
77.0
True
```

Chapter 6

a6.py

```
celsius = float(input("Temperature in Celsius: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"Fahrenheit: {fahrenheit:.1f}")
```

TERMINAL

```
> python a6.py
Temperature in Celsius: 18.5
Fahrenheit: 65.3
```

With 25 the output is Fahrenheit: 77.0.

Chapter 7

a7.py

```
name = "  sipho dlamini  "
name = name.strip().title()
print(name)
print(len(name))
print(name[0])
print(name[-1])
```

OUTPUT

```
Sipho Dlamini
13
S
i
```

Chapter 8

Run	Error (last line)	Fix
Run 1	IndentationError: unexpected indent	remove the spaces before <code>price</code>
Run 2	TypeError: can't multiply sequence by non-int of type 'float'	<code>units = int("8")</code>
Run 3	NameError: name 'totl' is not defined. Did you mean: 'total'?	<code>print(total)</code>

a8_fixed.py

```
units = int("8")
price = 12.5
total = units * price
print(total)
```

OUTPUT

```
100.0
```